

Automated Pre-Silicon Verification of High-Speed DDR5 and LPDDR5/6 Memory Controllers: Closed-Loop Timing, Mode Register, and PHY Synchronization in UVM

Manan Patel  (Member, IEEE) and Anirban Majumder  (Member, IEEE)

Google LLC, Mountain View, CA, USA — Google IT Services India Pvt. Ltd., Bengaluru, India

Abstract—External memory interfaces (such as LPDDR5/4 and DDR5) are essential components of modern mobile, cloud, and enterprise computing systems. While memory manufacturers focus on physical DRAM die development, the vast majority of semiconductor firms design custom application ASICs that require a dedicated Memory Controller to interface with these standardized external memories. Consequently, pre-silicon design verification of the Memory Controller RTL—acting as the Device Under Test (DUT) against a third-party DRAM Verification IP (VIP)—is a ubiquitous and critical challenge across the global semiconductor industry. This article presents an automated, pre-silicon configuration and closed-loop initialization framework for Memory Controller verification. The proposed solution parses JEDEC timing and configuration parameters directly from the DRAM VIP’s database files (such as Denali SOMA files) to configure the Memory Controller DUT’s registers, while a custom Mode Register Register Abstraction Layer (MR-RAL) tracks volatile DRAM VIP states in real-time. Furthermore, a dynamic, re-compilation-free PHY initialization flow randomizes interface parameters directly in the testbench, executes on-the-fly configuration generation via system calls, and parses the resulting register write sequences at runtime. By integrating these automated methodologies into pre-silicon verification flows, engineers can eliminate setup overhead, prevent false protocol violations, and enable comprehensive randomized testing of complex PHY and Memory Controller configurations.

Index Terms—Closed-loop initialization, DDR5, DFI 5.1, DRAM Verification IP (VIP), LPDDR5, LPDDR6, memory controller verification, Register Abstraction Layer (RAL), SOMA files, SystemVerilog, Universal Verification Methodology (UVM).

I. INTRODUCTION

External DRAM is an essential component of all modern mobile, cloud, and edge computing systems. While memory-die manufacturers focus on the physics and chemistry of memory cell arrays, the broader semiconductor industry focuses on designing custom application ASICs and System-on-Chips (SoCs) to accelerate specific workloads. To utilize standardized external DRAM (such as LPDDR5/4 or DDR5), these custom ASICs must integrate a dedicated Memory Controller. The Universal Verification Methodology (UVM) has become the standardized method for SoC verification, heavily relying on the integration of Verification IP (VIP) to model these complex external dependencies [1]. Validating these interfaces is increasingly complex (as summarized across JEDEC generations in Table I); for instance, LPDDR6 PHYs operating at 14,400 Mbps on advanced process nodes present severe signal integrity challenges, including simultaneous switching noise and inter-symbol interference caused by the simultaneous flipping of dozens of data lines on a parallel bus [2]. While physical signal integrity phenomena are validated using physical-level or SPICE-level simulations, the pre-silicon digital verification bottleneck lies in validating the complex Memory Reference Code (MRC) training algorithms that the digital controller executes to actively compensate for these channel distortions. Therefore, verifying the digital Memory Controller RTL against external DRAM protocol standards is a critical, universal bottleneck for almost every semiconductor company, except for specialized memory manufacturers [3]. Achieving successful Memory Reference Code (MRC) training requires utilizing specific controller logic to initiate transactions

on the LPDDR/DDR bus to tune sampling timings for all signals, scaling boot speeds from 4800 MT/s up to 14,400 MT/s [2, 4]. In these pre-silicon verification environments, teams pair the Memory Controller Device Under Test (DUT) with a third-party memory VIP to simulate physical DRAM chips. This interface is heavily standardized, often relying on documents like the DDR PHY Interface (DFI) Specification version 5.1, which describes signals, timing parameters, and training sequences for the interface between DDR memory controllers and PHY layers [5].

However, because JEDEC specifications (such as DDR5 JESD79-5 [6], LPDDR5 JESD209-5 [7], and LPDDR6 JESD209-6 [2]) outline hundreds of interdependent timing parameters that scale dynamically with clock frequency, synchronizing configurations between the Memory Controller DUT, the PHY, and the DRAM VIP remains a major industry challenge. Manually translating these dense, highly technical parameters across different simulation models introduces a massive risk of human error. A single parameter mismatch can trigger days of debugging overhead, masking actual design bugs behind false protocol violations. To address this, the industry requires a shift from static, manual configuration to automated, dynamic synchronization frameworks.

II. BACKGROUND AND CURRENT MEMORY CONTROLLER VERIFICATION CHALLENGES

In a classical pre-silicon verification flow, memory timing and configuration parameters must be manually extracted and programmed across multiple independent databases to align the Memory Controller DUT, PHY, and the DRAM VIP. This manual mapping is highly vulnerable to human error; a single tim-

TABLE I
GENERATIONAL EVOLUTION OF JEDEC DRAM ARCHITECTURE AND PRE-SILICON VERIFICATION COMPLEXITY (DDR4 TO LPDDR6)

Architectural / DV Metric	DDR4 (JESD79-4)	DDR5 (JESD79-5 [6])	LPDDR5/5T (JESD209-5 [7])	LPDDR6 (JESD209-6 [2])
Peak Data Rate per Pin	3,200 MT/s	6,400–8,800 MT/s	6,400–9,600 MT/s	10,667–14,400 MT/s
Channel Organization	1 × 64-bit	2 × 32-bit sub-channels	16-bit multi-channel	Dual 12-bit sub-channels (24-bit)
Mode Registers (MRs) per Rank	7 (MR0–MR6)	> 64 (MR0–MR63+)	> 64 (DVFS/FSP banked)	> 64 (Extended training/RAS MRs)
Frequency-Scaled Timing Params	~ 35 parameters	> 85 parameters	> 90 parameters	> 110 parameters
PHY Init / Training Registers	< 250 registers	> 1,200 registers	> 1,500 registers	> 2,000 registers
Classical Bring-Up per Speed-Bin	0.5–1 day (manual)	2–3 days (manual)	3–4 days (manual)	4–5+ days (manual)
Proposed Automated Flow	< 5 minutes	< 5 minutes	< 5 minutes	< 5 minutes

TABLE II
COMPARISON OF CLASSICAL VS. AUTOMATED MEMORY VERIFICATION FLOWS

Verification Metric	Classical Static Flow	Automated Closed-Loop Flow
Parameter Extraction	Manual mapping from datasheets	Automated script parsing
Re-compilation Requirement	Required for every parameter tweak	Eliminated (Run-time dynamic)
MR Synchronization	Manual database polling	Dynamic MR-RAL mirroring
Bring-up Overhead	Multiple days per speed-bin	< 5 minutes per speed-bin

ing parameter mismatch between the DRAM VIP simulation model [8] and the Memory Controller DUT's configuration registers can result in false protocol violations, consuming valuable debugging overhead.

Furthermore, high-speed memory PHY IPs require thousands of individual initialization and training parameters. In traditional verification flows, these parameters are often hardcoded directly into compiled SystemVerilog source files (such as packages or header files included via compiler directives) prior to compile-time. If a Design Verification (DV) engineer needs to modify a PHY setting—such as disabling a training stage or tweaking an impedance value—they must regenerate these source files and re-compile the entire simulation database. This compile-time dependency severely limits regression scaling and prevents the verification of dynamic, run-time parameter changes, as any parameter modification imposes a 100% recompilation penalty, grinding iterative debugging to a halt.

An additional critical challenge in memory controller verification involves the synchronization of DRAM Mode Register (MR) configurations. The MR configuration state must be seen identically by both the Memory Controller DUT RTL and the DRAM VIP. The traditional verification approach is highly cumbersome: engineers must manually retrieve MR-related configurations from UVM configuration databases or static files. This static, multi-step process is prone to synchronization lag and state mismatches, rendering the verification of dynamic training sequences highly error-prone. Table II illustrates the stark operational differences between this classical methodology and an automated closed-loop approach.

III. AUTOMATED SOMA-TO-REGISTER PARAMETER SYNCHRONIZATION

To eliminate manual mapping overhead, we implemented an automated pipeline that extracts JEDEC timing and configuration parameters directly from the DRAM VIP's Cadence Denali SOMA (Specification of Memory Architecture) files and programs them into the Memory Controller DUT. The simulation

behavior of a Cadence Denali VIP is largely dictated by these files; a `.denalirc` text file controls general simulation flow, while the SOMA file defines protocol standards and drivable values [9]. Because SOMA is a Cadence-standard format used to parameterize verification models, it serves as an authoritative, machine-readable source of truth for all memory timing requirements, including complex parameters such as CAS latency, row precharge time (t_{RP}), and RAS-to-CAS delay (t_{RCD}) [6–8].

During the pre-compilation phase (integrated into the build flow via `Makefile`), an automated script parses these DRAM VIP SOMA files. To ensure the tools can locate these essential parameters, the simulation environment must properly configure the `$DENALI` environment variable to point to the correct VIP root directory. SOMA files contain certified timing specifications expressed in physical time units (picoseconds or nanoseconds) as well as clock cycles. Because the Memory Controller RTL operates on a digital clock domain, any parameter defined in physical time must be strictly converted into discrete clock cycles before it can be programmed into the DUT registers. Furthermore, the parsing script is designed to handle various speed-bins automatically, ensuring that when the simulation frequency changes, all dependent timing parameters are dynamically recalculated without requiring manual intervention.

For parameters defined in physical time units, the utility normalizes the values and translates them into clock cycles based on the active simulation frequency using JEDEC rounding conventions. This translation is mathematically expressed in (1), where the resulting cycle count is always rounded up to ensure timing constraints are never violated by being too short:

$$N_{\text{cycles}} = \lceil T_{\text{param}} \times f_{\text{CLK}} \rceil. \quad (1)$$

Here, N_{cycles} represents the calculated cycle count, T_{param} is the JEDEC parameter in physical time, and f_{CLK} is the active clock frequency.

For parameters that are already defined in clock cycles, the script bypasses this translation and passes the values directly. The resolved clock cycles are then automatically mapped to their respective memory-mapped registers inside the Memory Controller DUT, generating a SystemVerilog configuration package that is subsequently compiled and elaborated. This ensures that the Memory Controller DUT, the physical layer, and the DRAM VIP operate in perfect clock-cycle synchronization, preventing false timing drift failures and completely automating the initial parameter programming phase. By automating this SOMA-to-register synchronization, verification engineers can seamlessly switch between different memory speed grades, such as moving from LPDDR5 4800 MT/s to 9600 MT/s in LPDDR5T, simply

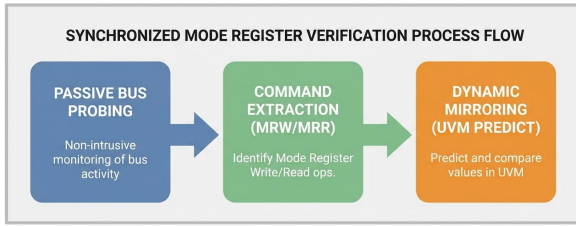


Fig. 1. Automated Mode Register (MR) tracking and dynamic mirroring flow.

by pointing the environment to a new SOMA file. This eliminates the days of manual datasheet transcription previously required for each new configuration.

Additionally, most DRAM vendors maintain SOMA configurations for their custom requirements, especially in cases where they deviate from the baseline JEDEC specifications. The automated SOMA-to-register configuration flow allows the DV environment to seamlessly migrate to vendor-specific timings.

IV. SYNCHRONIZED MODE REGISTER (MR) VERIFICATION AND MODELING

To resolve the state-matching bottleneck across DRAM boundaries, we architected a unified Mode Register read/write API and a customized Mode Register Register Abstraction Layer (MR-RAL) model to align the Memory Controller DUT and the DRAM VIP. Volatile register fields undergo hardware-driven changes independently of bus transactions, rendering them invisible to standard UVM prediction logic and necessitating active monitoring to keep the register abstraction layer synchronized [10]. Because DRAM Mode Registers are inherently volatile during initialization and training sequences, standard frontdoor polling is vastly insufficient.

The unified MR API exposes a set of tasks that programmatically handle and verify MR reads and writes. Instead of manually extracting configurations, this API manages transaction generation and automatically synchronizes the MR states between the Memory Controller DUT RTL and the DRAM VIP. To track these volatile DRAM states on-the-fly, a dedicated register group within the UVM RAL block is paired with a passive interface monitor. In UVM, the passive prediction mode updates the register model using transactions captured by the Universal Verification Component (UVC) monitor without connecting the model to the UVC sequencer [1]. This allows the testbench to silently observe the bus.

The synchronization process operates in three distinct phases, as depicted in Fig. 1. First, through *Passive Bus Probing*, the UVM monitor actively probes the physical interface bus to spy on transactions between the Memory Controller DUT and the DRAM VIP. To support complex multi-rank and multi-channel DDR5 topologies, the monitor also extracts active chip select (CS) and channel-routing signals. The passive predictor uses this routing metadata to map the extracted MR command to the corresponding rank-specific or channel-specific register block inside the hierarchical MR-RAL model. Second, during *Command Extraction*, when the monitor detects a Mode Register Write (MRW) or Read (MRR) command, it extracts the target register address and the transferred data payload.

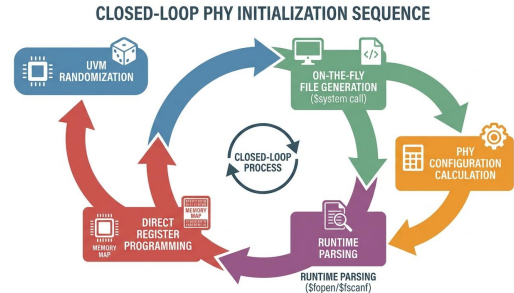


Fig. 2. Closed-loop PHY initialization execution sequence.

Finally, through *Dynamic Mirroring*, explicit prediction is utilized. While auto-prediction updates mirror values during frontdoor access, explicit prediction uses a `uvm_reg_predictor` component connected to the monitor's analysis port to keep the register model updated [1]. In standard explicit prediction, the `uvm_reg_predictor` receives transactions from the monitor's analysis port, translates them using an adapter, and internally invokes the register's `predict()` method. Alternatively, for custom backdoor mirroring, the testbench can bypass standard prediction blocks by calling the `predict()` method directly:

```
mr_ral_block.MR_reg[addr].predict(payload, .kind(UVM_PREDICT_DIRECT));
```

This combination of a synchronized MR task API and dynamic MR-RAL mirroring ensures that check monitors and UVM scoreboards can query the exact state of the DRAM VIP at any moment. To maintain state alignment during hardware-driven asynchronous or system resets—which restore default register values without issuing bus transactions—the MR-RAL model is bound to the system reset signal. Upon reset assertion, the passive predictor triggers a reset of the RAL block, ensuring perfect state alignment is maintained across power cycles. Because these backdoor updates occur in zero simulation time, false out-of-sync checker flags are entirely eliminated, yielding a massive improvement in verification accuracy.

V. RE-COMPILE-FREE PHY INITIALIZATION AND RUNTIME OVERRIDES

To eliminate static pre-compiled configuration files and enable thorough randomized testing of the Memory Controller DUT, we developed an automated, closed-loop programmatic initialization flow. A standard SystemVerilog and UVM environment typically uses a Sequencer to generate transaction scenarios—such as testing boundary addresses and resets—which a Driver then maps to the DUT [1]. However, there may be cases where the DV testbench must be integrated with third-party IPs that use a separate compile or execution flow that is not inherently compatible with SystemVerilog/UVM. An example may be the usage of a third-party PHY IP, which requires a pre-compilation to generate static initialization sequences. Applying the standard UVM-based randomized testing architecture to thousands of static PHY initialization variables creates a massive performance bottleneck. Utilizing the standard UVM configuration database (`uvm_config_db` API) for run-time updates introduces significant simulation overhead because it requires costly string lookup operations for hundreds of elements [11].

Therefore, the configuration process is driven directly from within the SystemVerilog/UVM testbench using a dynamic execution sequence that completely bypasses the standard UVM configuration database for PHY initialization. First, through

UVM Parameter Randomization, high-level PHY physical features (such as training flags, impedance settings, and target clock frequencies) are randomized using standard SystemVerilog architecture constraints.

Second, as illustrated in Fig. 2, during *On-the-Fly File Generation*, the testbench writes these randomized values to a temporary configuration seed and uses a SystemVerilog system call (\$system) to trigger an external configuration compiler utility. To ensure file isolation during concurrent regressions on a parallel compute cluster, the testbench appends the simulation process ID (PID) or a unique seed hash to the temporary file names, preventing race conditions and file collisions. Third, for *PHY Configuration Calculation*, the utility runs on-the-fly to generate a uniquely isolated, vendor-compliant initialization text file. Fourth, a specialized SystemVerilog parsing function opens and reads this generated text file at runtime using standard file I/O operations (\$fopen, \$fscanf), extracting the hex register write sequence (addresses and data payloads). To guarantee execution robustness, the parsing logic evaluates the return status of the \$system call and validates the returned file descriptor from \$fopen before parsing, immediately throwing a UVM error if the external utility fails or the file is unreadable.

Finally, through *Direct Register Programming*, the testbench applies the extracted sequence directly to program the active PHY RTL/model registers. This mechanism acts similarly to a pre-runtime signal forcing mechanism, allowing specific internal register states to be initialized before simulation begins, which synchronizes initial conditions across tests and bypasses standard protocol runtime overhead [11]. This closed-loop programmatic initialization bypasses static file generation entirely. Any PHY parameter or training threshold can be randomized and re-applied at run-time without requiring testbench re-compilation, representing a major efficiency improvement.

VI. CONCLUSION AND INDUSTRY IMPLICATIONS

Implementing this automated parameter synchronization and dynamic initialization framework yields substantial productivity gains in Memory Controller verification. The automated timing pipeline reduces the bring-up time for new memory speedbin configurations on the Memory Controller DUT from several days to under a few minutes subject to initial parser setup and register-map mapping configurations. By extracting JEDEC parameters directly from authoritative SOMA files, semiconductor teams can significantly mitigate the human error associated with manual datasheet transcription, saving countless hours previously lost to debugging false protocol violations. While the current pipeline is designed around the proprietary Cadence Denali SOMA format, the underlying parameter synchronization methodology is generalizable. To adapt this framework for Verification IPs from other major vendors (such as Synopsys or Siemens), the front-end parser can be adjusted to read standardized XML schemas like IP-XACT [12], leaving the core cycle-conversion and register-mapping logic intact.

By replacing pre-compiled static files with a dynamic system-call and runtime parsing flow, re-compilation bottlenecks are completely eliminated. This allows automated regressions to sweep hundreds of randomized physical configurations in a single run, rather than requiring engineers to manually re-compile the simulation environment for every minor impedance or timing tweak. The introduction of the MR-RAL passive tracking

mechanism further guarantees that the verification environment maintains perfect state synchronization with the volatile DRAM VIP in zero simulation time.

Ultimately, these automated methodologies provide a highly scalable, robust blueprint that improves verification efficiency, enhances regression flexibility, and establishes a robust verification standard for high-bandwidth memory interfaces. As external memory standards like DDR5 and LPDDR5 continue to push the physical limits of signal integrity and timing margins, removing static verification bottlenecks will be paramount for delivering custom ASICs and SoCs to market on schedule.

REFERENCES

- [1] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020 (Revision of IEEE Std 1800.2-2017), Sep. 2020, doi: 10.1109/IEEESTD.2020.9195920.
- [2] *Low Power Double Data Rate 6 (LPDDR6) Standard*, JEDEC Standard JESD209-6, JEDEC Solid State Technology Association, Arlington, VA, USA, 2025.
- [3] L. Steiner, C. Sudarshan, M. Jung, D. Stoffel, and N. Wehn, "A framework for formal verification of DRAM controllers," in *Proc. ACM/IEEE Int. Symp. Memory Systems (MEMSYS)*, Oct. 2022, pp. 1–7, doi: 10.48550/arXiv.2209.14021.
- [4] L. Steiner, M. Jung, M. Huonker, and N. Wehn, "Unveiling the real performance of LPDDR5 memories," in *Proc. ACM/IEEE Int. Symp. Memory Systems (MEMSYS)*, Oct. 2022, Art. no. 9, pp. 1–3, doi: 10.48550/arXiv.2209.14756.
- [5] *DFI 5.1 Specification: DDR PHY Interface*, DDR PHY Interface (DFI) Group, May 2021.
- [6] *DDR5 SDRAM Standard*, JEDEC Standard JESD79-5, JEDEC Solid State Technology Association, Arlington, VA, USA, 2020.
- [7] *Low Power Double Data Rate 5 (LPDDR5) Standard*, JEDEC Standard JESD209-5, JEDEC Solid State Technology Association, Arlington, VA, USA, 2019.
- [8] L. Steiner, M. Jung, F. S. Prado, K. Bykov, and N. Wehn, "DRAMSys4.0: An open-source simulation framework for in-depth DRAM analyses," *Int. J. Parallel Program.*, vol. 50, no. 2, pp. 217–242, Apr. 2022, doi: 10.1007/s10766-021-00723-5.
- [9] Y.-N. Yun, J.-B. Kim, N.-D. Kim, and B. Min, "UVM-based verification of DDR4 SDRAM controller," in *Proc. IEEE Int. SoC Design Conf. (ISOC)*, Jeju, South Korea, Oct. 2021, pp. 305–306, doi: 10.1109/ISOC53507.2021.9613965.
- [10] M. Litterick and M. Harnisch, "Advanced UVM register modeling: There's more than one way to skin a reg," in *Proc. Design Verification Conf. (DVCon)*, San Jose, CA, USA, 2012, pp. 1–11.
- [11] P. Marriott and M. Ronan, "Run-time configuration of a verification environment: A novel use of the OVM/UVM analysis pattern," in *Proc. Design Verification Conf. (DVCon)*, San Jose, CA, USA, 2013, pp. 1–8.
- [12] *IEEE Standard for IP-XACT, Standard Structure for Packaging, Integrating, and Reusing IP within Tool Flows*, IEEE Std 1685-2022 (Revision of IEEE Std 1685-2014), Feb. 2023, doi: 10.1109/IEEESTD.2023.10044519.

Author Biographies

Manan Patel (Member, IEEE) received his B.E. degree in Electronics and Communication from Gujarat University, India, in 2009, and his M.Tech. degree in VLSI Design from Nirma University, India, in 2013. He is currently a Senior ASIC Design Verification Engineer at Google, USA, working on Quantum AI control electronics. Previously, he held design verification and verification IP (VIP) development roles at Qualcomm, Cadence, and STMicroelectronics. His research interests include high-speed memory subsystems, platform security verification, and automated verification methodologies. ORCID ID: 0009-0004-0290-6225.

Anirban Majumder (Member, IEEE) graduated with a B.Tech. and M.Tech. dual degree in Electrical Engineering from the Indian Institute of Technology Kharagpur, India, in 2022, along with a minor in Computer Science and Engineering. He is currently a Silicon Engineer at Google IT Services India Pvt. Ltd. with four years of experience in DDR and LPDDR technologies. He is also a Member of IEEE since 2026. ORCID ID: 0009-0002-3490-1395.